

Python Code

```

1
2
3 #####3
4 class Graph():
5     def __init__(self,no_nodes):
6         self.graph=[]
7         for i in range(no_nodes):
8             L=[]
9             for j in range(no_nodes):
10                 L.append(0)
11             self.graph.append(L)
12     def add_edge(self,node1,node2):
13         self.graph[node1][node2]=1
14         self.graph[node2][node1]=1
15
16     def remove_edge(self,node1,node2):
17         self.graph[node1][node2]=0
18         self.graph[node2][node1]=0
19
20     def add_vertex(self):
21         L=[]
22         for j in range(len(self.graph)):
23             L.append(0)
24         self.graph.append(L)
25         for i in self.graph:
26             i.append(0)
27     # def remove_vertex(self,node):
28     #     self.graph.pop(node)
29     #     for i in self.graph:
30     #         i.pop(node)
31
32
33 print("ADJ_matrix")
34
35 g=Graph(4)
36 g.add_edge(1,2)
37 g.add_edge(2,3)
38 g.add_edge(0,1)
39 g.add_edge(0,3)
40 # g.remove_vertex(3)
41 # g.remove_vertex(3)
42 # g.add_vertex()
43 # g.remove_edge(2,3)
44 for i in g.graph:
45     print(i)
46
47
48 #####
49 class Graph():
50     def __init__(self,node_nodes):
51         self.nodes=node_nodes
52         self.graph=[[ for i in range(self.nodes)]]

```

```

53
54     def add_edge(self, node1, node2):
55         self.graph[node1].append(node2)
56         self.graph[node2].append(node1)
57
58     def remove_edge(self, node1, node2):
59         self.graph[node1].remove(node2)
60         self.graph[node2].remove(node1)
61
62     def add_vertex(self):
63         self.graph.append([])
64
65     # def remove_vertex(self, node):
66     #     for i in self.graph:
67     #         if node in i:
68     #             i.remove(node)
69     #     if node!=self.graph[-1]:
70     #         for i in range(len(self.graph)):
71     #             for j in range(len(self.graph[i])):
72     #                 if self.graph[i][j]!=0:
73     #                     self.graph[i][j]-=1
74
75     #     self.graph.pop(node)
76
77
78
79     print("ADJ_list")
80     g=Graph(6)
81     g.add_edge(0,4)
82     g.add_edge(0,3)
83     g.add_edge(1,3)
84     g.add_edge(2,4)
85     g.add_edge(2,5)
86     g.add_edge(5,1)
87
88     for i in g.graph:
89         print(i)
90     # g.remove_vertex(5)
91     for i in g.graph:
92         print(i)
93     #####
94     class Graph():
95         def __init__(self, no_nodes, nature):
96             self.nature = nature
97             self.no=no_nodes
98             self.graph={i:[] for i in range(self.no)}
99
100         def add_edge(self, node1, node2):
101
102             if self.nature == "directed":
103                 self.graph[node1].append(node2)
104             else:
105                 self.graph[node1].append(node2)
106                 self.graph[node2].append(node1)
107

```

```

108     def remove_edge(self,node1,node2):
109         if self.nature == "directed":
110             self.graph[node1].remove(node2)
111         else:
112             self.graph[node1].remove(node2)
113             self.graph[node2].remove(node1)
114
115     def add_vertex(self):
116         self.graph[len(self.graph)]=[]
117
118     # def remove_vertex(self,node):
119     #     for i in self.graph:
120     #         if node in self.graph[i]:
121     #             self.graph[i].remove(node)
122
123     #     pass
124
125     print("ADJ_dict")
126
127     g=Graph(6,"directed")
128     # g.add_edge()
129     g.add_edge(2,4)
130     g.add_edge(0,5)
131     g.remove_edge(0,5)
132     g.add_vertex()
133     print(g.graph)
134     #####333
135     # Roll numbers :
136     # Suleman Khokhar 281152801
137     # Harris Parvez 281133450
138
139
140     class Graph:
141
142         '''Should initialize the adjacency matrix and
143         store number of nodes and if the graph is directed or not.'''
144
145         def __init__(self, num_of_nodes, directed=True):
146             self.num_of_nodes = num_of_nodes+1
147             self.directed = directed
148             self.graph = [[0 for j in range(self.num_of_nodes)] for i in range
149 (self.num_of_nodes)]
150
151             #should add an edge between node1 and node2
152             def add_edge(self, node1, node2, weight=1):
153                 if 0<=node1 < self.num_of_nodes and 0<=node2 < self.num_of_nodes:
154                     self.graph[node1][node2] = weight
155                     if not self.directed :
156                         self.graph[node2][node1] = weight
157                     # else:
158                     #     print("invalid")
159
160             #should print
161             def print_adj_matrix(self):
162                 for i in self.graph:

```

```

162         for j in i:
163             print(f"{j} ",end="")
164         print()
165     #should remove an edge between node1 and node2
166     def remove_edge(self, node1, node2, weight=1):
167         if 0<=node1 < self.num_of_nodes and 0<=node2 < self.num_of_nodes:
168             self.graph[node1][node2] = 0
169             if not self.directed :
170                 self.graph[node2][node1] = 0
171
172     #should add a new node
173     def addVertex(self, node):
174         self.num_of_nodes+=1
175         for row in self.graph:
176             row.append(0)
177         self.graph.append([0]*self.num_of_nodes)
178
179     #should remove the node as well all edges incident with it.
180     def removeVertex(self, node):
181         if 0 <= node < self.num_of_nodes:
182             self.graph.pop(node)
183             for row in self.graph:
184                 row.pop(node)
185             self.num_of_nodes -= 1
186
187     def printing(self):
188         for i in self.graph:
189             for j in i:
190                 print(f"{j} ",end="")
191             print()
192     def bfs(self, start_node):
193         visited = []
194         queue = []
195
196         # Start from the given node
197         visited.append(start_node)
198         queue.append(start_node)
199
200         while queue:
201             # Pop the first element to simulate a queue (FIFO)
202             current = queue.pop(0)
203
204             # Visit all adjacent nodes
205             for neighbor in range(self.num_of_nodes):
206                 if self.graph[current][neighbor] != 0 and neighbor not in v
isited:
207                     visited.append(neighbor)
208                     queue.append(neighbor)
209
210             return visited
211
212         # DFS using stack (iterative)
213     def dfs_stack(self, start_node):
214         visited = []
215         stack = [start_node]

```

```

216
217         while stack:
218             current = stack.pop()          # pop last (LIFO)
219
220             if current not in visited:
221                 visited.append(current)
222
223                 # Push neighbors in reverse order to keep order correct
224                 for neighbor in range(self.num_of_nodes - 1, -1, -1):
225                     if self.graph[current][neighbor] != 0 and neighbor not
in visited:
226                         stack.append(neighbor)
227
228             return visited
229     # DFS using recursion
230     def dfs_recursive(self, start_node):
231         visited = []
232         self._dfs_helper(start_node, visited)
233         return visited
234
235     def _dfs_helper(self, node, visited):
236         visited.append(node)
237
238         for neighbor in range(self.num_of_nodes):
239             if self.graph[node][neighbor] != 0 and neighbor not in visited:
240                 self._dfs_helper(neighbor, visited)
241
242
243     def path_exists(self, node1, node2):
244         '''Return True if there is a path from node1 to node2, False otherw
ise.'''
245
246         if node1 < 0 or node1 >= self.num_of_nodes or node2 < 0 or node2 >=
self.num_of_nodes:
247             print("Invalid nodes.")
248             return False
249         visited = []
250         queue = []
251         visited.append(node1)
252         queue.append(node1)
253         while queue:
254             current = queue.pop(0)
255             if current == node2:
256                 return True
257             for neighbor in range(self.num_of_nodes):
258                 if self.graph[current][neighbor] != 0 and neighbor not in v
isited:
259                     visited.append(neighbor)
260                     queue.append(neighbor)
261             return False
262
263
264     # number_of_egdes = int(input("Enter the number of edges : "))
265     # direct = bool(input("Enter type of graph : "))
266     graph = Graph(8,False)

```

```

267 # graph = Graph(number_of_egdes,direct)
268
269
270 # graph.printing()
271
272 print("_____adding  edgdes _____")
273 graph.add_edge(0,1,3)
274 graph.add_edge(0,3,2)
275 graph.add_edge(0,8,4)
276 graph.add_edge(1,7,4)
277 graph.add_edge(2,7,2)
278 graph.add_edge(2,3,6)
279 graph.add_edge(2,5,1)
280 graph.add_edge(3,2,6)
281 graph.add_edge(3,4,1)
282 graph.add_edge(3,0,2)
283 graph.add_edge(4,3,1)
284 graph.add_edge(4,8,8)
285 graph.add_edge(5,2,1)
286 graph.add_edge(5,6,8)
287 graph.add_edge(6,5,8)
288 graph.add_edge(7,2,2)
289 graph.add_edge(7,1,4)
290 graph.add_edge(8,0,4)
291 graph.add_edge(8,4,8)
292
293 print("_____printing edgdes _____")
294
295 graph.printing()
296
297 print("_____
")
298
299 # # graph.addVertex(9)
300 # print("_____printing edgdes _____")
301
302 # graph.printing()
303
304 # print("_____
")
305 # print()
306 # print("Removing egde ")
307 # graph.removeVertex(3)
308
309 # print("_____printing edgdes _____")
310
311 # graph.printing()
312
313 # print("_____
")
314 # _____
315
316 result = graph.bfs(0)
317 print("_____")

```

```
318 print(result)
319 print(graph.path_exists(4,3))
320
321 print("DFS (Stack):", graph.dfs_stack(0))
322 print("DFS (Recursive):", graph.dfs_recursive(0))
323
```